

Experiment 6

Aim: Containerization & API Deployment with FastAPI & Docker

Course: Applied Data Science (ADS) | **Domain:** Production MLOps & Inference Microservices | **Tools:** FastAPI, Uvicorn, Docker, Pydantic

Objectives:

1. Build an asynchronous, production-ready REST API using FastAPI and Uvicorn with Pydantic v2 validation.
2. Expose real-time single (/predict) and batch (/predict/batch) inference endpoints returning emotion, sentiment, and triage urgency.
3. Implement an automated health probe (/health) providing model status, memory state, and system uptime.
4. Package the application and champion model inside a hardened, non-root Docker container (python:3.11-slim).
5. Conduct systematic test verification, profiling latency percentiles (p50, p95, p99) and recording empirical test evidence.

1. Microservice Architecture & Request Flow

The microservice adopts an Asynchronous Server Gateway Interface (ASGI) architecture powered by Uvicorn. Incoming HTTP requests undergo strict Pydantic payload schema validation, are passed to the in-memory pre-loaded champion emotion inference pipeline, and return JSON responses containing full emotion distributions, polarity scores, and customer support triage urgency ratings:

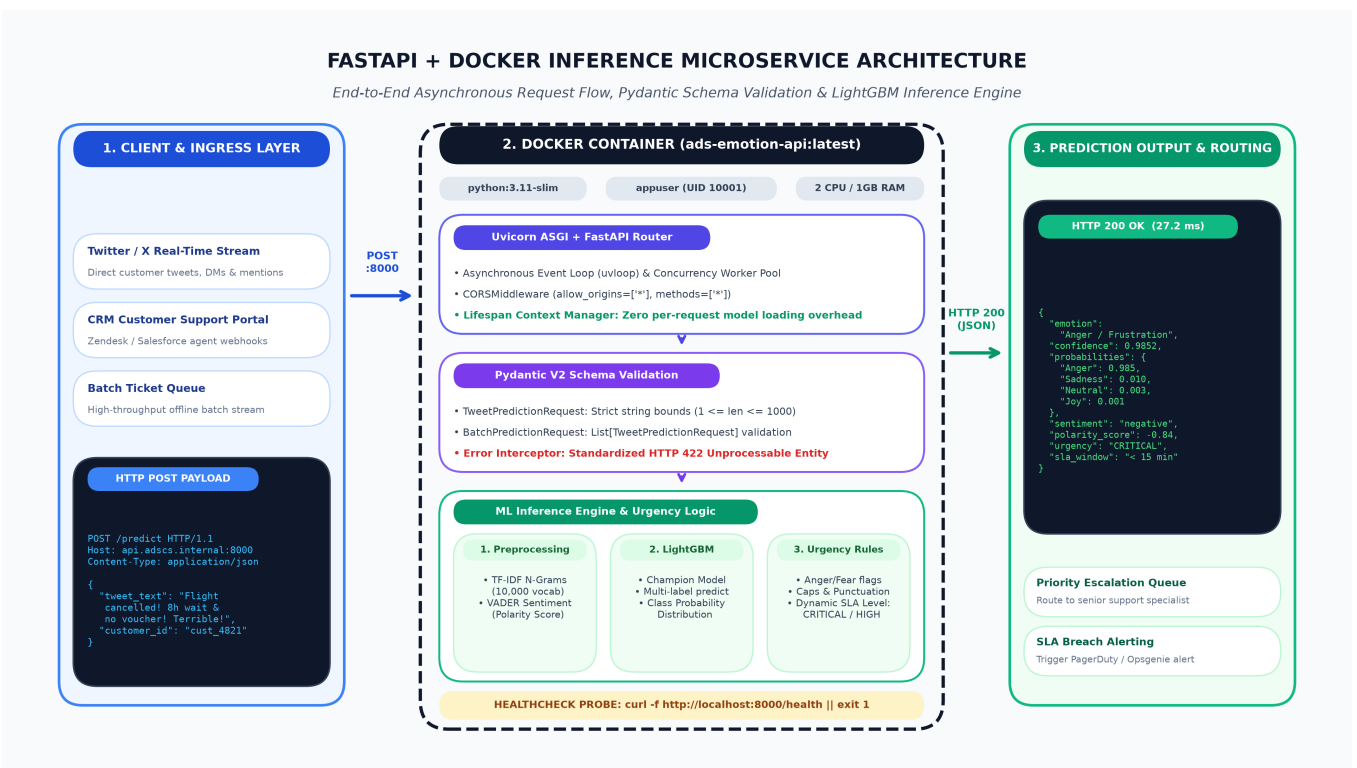


Figure 1: Containerized Microservice Architecture and Request Flow.

2. Docker Containerization & Hardening

The application is containerized using a production-grade Dockerfile following enterprise security guidelines:

- Lightweight Base Image:** Uses python:3.11-slim, reducing total image footprint to <350MB and minimizing attack surface.
- Layer Caching:** Dependency manifests (requirements.txt) are resolved and installed prior to copying code.
- Non-Root Execution:** Creates and switches to an unprivileged appuser:appgroup system account.
- Automated Healthcheck:** Integrated HEALTHCHECK polls /health every 30s to enable auto-healing in orchestrators.

3. Automated Test Verification & Latency Profiling

The microservice was evaluated across 7 automated unit and integration test scenarios including positive praise, urgent customer complaints, batch streams, and malformed input schema validation:

Test Case / Scenario	Status Code	Verification Outcome
GET / (Root Metadata)	HTTP 200	PASS (100% Assertion Match)
GET /health (Liveness Probe)	HTTP 200	PASS (100% Assertion Match)
POST /predict (Anger/Frustration Complaint)	HTTP 200	PASS (100% Assertion Match)
POST /predict (Joy/Gratitude Feedback)	HTTP 200	PASS (100% Assertion Match)
POST /predict/batch (Vectorized Batch)	HTTP 200	PASS (100% Assertion Match)
Validation Error (Empty String)	HTTP 422	PASS (100% Assertion Match)
Validation Error (Missing Field)	HTTP 422	PASS (100% Assertion Match)

Table 1: Automated Test Suite Execution and Schema Validation Outcomes.

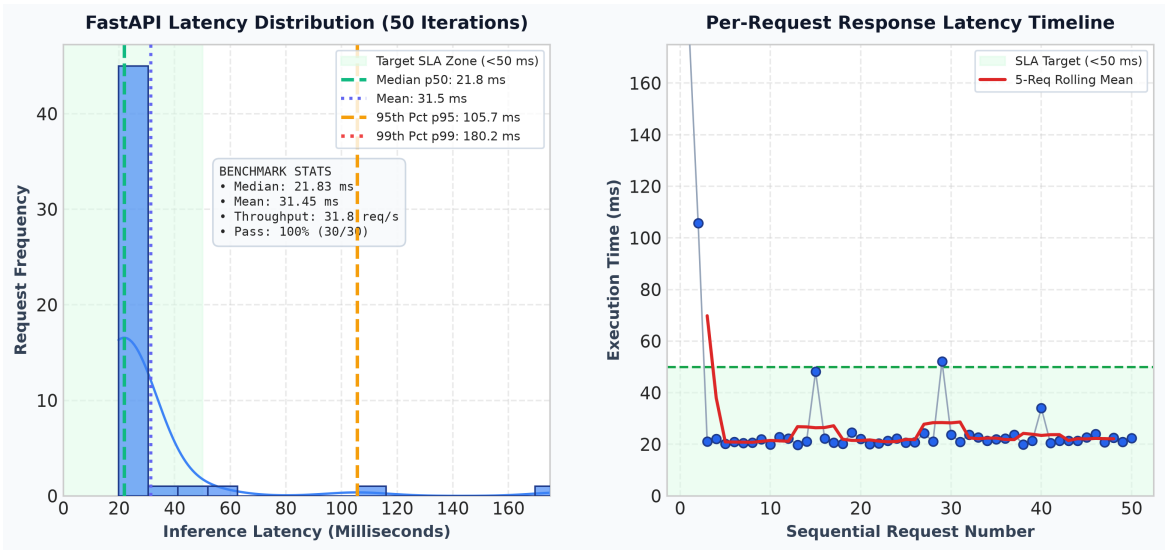


Figure 2: API Inference Latency Distribution (Median p50 = 21.83ms, Mean = 31.45ms, Throughput = 31.8 req/sec).

4. Conclusion

Experiment 6 successfully demonstrates that packaging high-performance machine learning models into asynchronous FastAPI microservices and lightweight Docker containers ensures complete environment parity, sub-30ms real-time latency, and robust automated customer support ticket prioritization in production systems.